

The Nearest-the-Pin Parimutuel

A continuous, projection-scored pool for distributional forecasts

Peter Cotton · *Working draft v0.2* · 2026

Status. This is an evolving working note, not a finished paper. The core mechanism and the projection identity (§4) are implemented and unit-tested in [mechanisms/nearest_the_pin.py](#); the connections to the random-projections literature (§4) and the Schur pseudo-likelihood (§5) are sketched and flagged where conjectural. *v0.2*: added the §3 caveat that truthful reporting concerns the density, not the cloud — under KDE settlement at the raw outcome the optimal cloud is the belief deconvolved by the bandwidth; see the companion paper [Scoring Point-Cloud Distributional Submissions](#).

Abstract

The parimutuel is the oldest information-aggregation mechanism: bettors pool stakes on a finite set of outcomes and winners split the pot pro rata. We study its limit when the outcome is a point $z \in \mathbb{R}^d$ and each participant submits a *predictive density* — in practice a cloud of Monte-Carlo samples. The pot is split in proportion to the density a participant placed at the realised z , relative to the crowd: a nearest-the-pin reward. We make three points. (i) The mechanism is self-funding and, for a log-wealth (Kelly) maximiser, strictly incentive-compatible: the unique optimal report is one's true density, which connects the pool's growth-optimal strategy to the logarithmic scoring rule. (ii) In high dimensions the joint density is hard to estimate; the projection version — used at [monteprediction.com](#) for eleven-dimensional forecasts — replaces it with random one-dimensional projections, and by the energy-distance projection identity the average sliced CRPS recovers the multivariate energy score exactly (up to a dimension constant). This places the mechanism inside the *random-projections* literature. (iii) The high-dimensional scoring problem is the same one solved, from the density side, by the Schur pseudo-likelihood: nearest-the-pin sits between two strategies for scoring a joint forecast when d is large relative to the data — structured density estimation (Schur/Vecchia) and random projection (energy/sliced scores).

1. From the racetrack to a cloud of points

A classical parimutuel (§[parimutuel](#)) operates over a finite partition of outcomes $\{1, \dots, n\}$. Bettors stake W_i on outcomes; if outcome j realises, the pot is divided among backers of j in proportion to their stake. The implied probabilities are the pool fractions, and — crucially — the operator bears no risk.

Modern forecasting is rarely categorical. The object of interest is a *full predictive distribution* over a continuous, often multivariate, quantity: next week's joint returns of eleven sector ETFs, a spatial field, a vector of demands. The natural generalisation of the parimutuel replaces “a ticket on outcome j ” with “probability mass placed near the point z ”, and the pro-rata split

with a split proportional to the density each participant ascribed to the realised z . This is the reward rule of monteprediction.com, described there as

“a splitting of the pot in proportion to the density that you ascribe to the truth z ,
[which] also depends on the density that others ascribe to z ,”

with participants risking a fixed fraction (≈ 0.1) of their wealth each round. We call it the nearest-the-pin parimutuel: the closer your mass lands to the pin, relative to the field, the larger your share of the pot.

2. The discrete parimutuel is log-optimally truthful

Why a *density* split, and not some other functional? Because the parimutuel already has a sharp incentive theory in the discrete case, and it is exactly the one we want.

Consider n outcomes with true probabilities p_k , and a parimutuel in which a player allocates a unit stake as a distribution $b = (b_1, \dots, b_n)$ over outcomes. If the rest of the pool's stake fractions are r_k and the player is small, a unit bet on k returns $1/r_k$ when k occurs. A player maximising the expected log growth of wealth solves

$$\max_{b \in \Delta} \sum_k p_k \log \frac{b_k}{r_k},$$

whose unique maximiser is $b_k = p_k$ — *bet your beliefs* (Kelly 1956; Breiman 1961). The growth rate at the optimum is the Kullback–Leibler divergence $D(p \parallel r)$, i.e. the player profits exactly to the extent their (true) belief beats the crowd's implied distribution. Log-wealth maximisation and truthful reporting coincide, and the quantity being maximised is the logarithmic score of the player's report against the realised outcome, net of the crowd.

This is the discrete shadow of the continuous mechanism, and it tells us the continuous split should be proportional to density (the continuous analogue of b_k) and the natural objective is log-wealth growth.

3. The nearest-the-pin parimutuel

Mechanism. Each of n participants holds wealth W_i and submits a predictive density q_i over $\mathbb{R}^d$ (in practice, a sample cloud, from which q_i is recovered by kernel density estimation; [kde_density](#)). Each risks a stake $s_i = b W_i$ for a fixed fraction $b \in (0, 1)$. The outcome z is revealed. The collected pot $S = \sum_i s_i$ is redistributed in proportion to $s_i q_i(z)$ — stake times the density placed at z — so participant i 's wealth change is

$$\Delta W_i = S \frac{s_i q_i(z)}{\sum_j s_j q_j(z)} - s_i \quad (\text{NTP})$$

Self-funding. $\sum_i \Delta W_i = S - S = 0$: the pool is a pure transfer, the operator bears no risk, exactly as in the racetrack tote. (Implemented and tested in [pot_split](#).)

Truthfulness. Fixing the field’s reports and stakes, the aggregate density at z is $Q(z) = \sum_j s_j q_j(z)$. A small participant’s expected log-wealth growth from reporting q is, to first order in b ,

$$\mathbb{E}_{z \sim p} \left[\log \left(1 + b \left(\frac{S}{s_i} \frac{q(z)}{Q(z)} - 1 \right) \right) \right] \approx b \left(\mathbb{E}_{z \sim p} \left[\frac{S q(z)}{Q(z)} \right] - 1 \right),$$

and the report q maximising $\mathbb{E}_{z \sim p} [q(z)/Q(z)]$ subject to $\int q = 1$ is, by the same Gibbs argument as §2, $q = p$: the true density. As in the discrete case the growth rate is governed by a logarithmic comparison of the player’s density to the crowd’s. We verify the incentive numerically in `test_nearest_the_pin.py`: a truthful reporter out-grows a biased one against a truthful field.

Caveat (added v0.2): the truthful object is the *density*, not the *cloud*. The claim above concerns the reported density q . When q is *constructed from a sample cloud by KDE* — the parenthetical in the Mechanism paragraph, and what `pot_split` actually does — the report space is the cloud, the mechanism applies $q = \rho * \varphi_h$, and the optimal cloud is *not* drawn from one’s belief: it is the belief deconvolved by the bandwidth (Gaussian belief $N(\mu, \tau^2)$, bandwidth $h \square$ optimal cloud $N(\mu, \tau^2 - h^2)$). The two statements are consistent — the optimal *smoothed* cloud still equals the true density, so the density-level optimum stands — but “submit samples from your belief” is not optimal under raw-outcome settlement. Truthfulness of the cloud itself is restored by jittering the pin by the KDE kernel, strictly proper via injectivity of Gaussian convolution. See the companion paper, *Scoring Point-Cloud Distributional Submissions*, and `mollified_log_score` in `nearest_the_pin.py`.

Relationship to other mechanisms. - It is the continuous, density-weighted limit of the parimutuel (§2), and a density-pot-split generalisation of Pennock’s dynamic parimutuel market (DPM): the DPM prices shares on discrete outcomes via a cost function; NTP prices density mass on a continuum. - Its truthfulness rests on the logarithmic score / log-wealth growth, the same object that, applied *sequentially*, gives Hanson’s LMSR. NTP is the *pooled* reading; LMSR is the *sequential* reading. - The score it implicitly applies to a sample cloud is a strictly proper scoring rule for the predictive density; §4 makes this precise and connects it to the energy score.

4. The projection version

The high-dimensional obstruction. In $d = 11$ dimensions, recovering a participant’s density $q_i(z)$ from a finite sample cloud by KDE is fragile: the bandwidth, the curse of dimensionality, and the heavy tails of financial returns all bite. `monteprediction`’s contest has participants submit *a million* scenarios precisely because dense coverage is needed to pin down $q_i(z)$. Even so, a density estimate in eleven dimensions from finite samples is a delicate object.

Slicing. The projection (sliced) version sidesteps the d -dimensional density. Draw random unit directions $u \in S^{d-1}$, project every participant’s cloud and the outcome onto each u , and score the resulting *one-dimensional* forecasts — where density estimation, the CRPS, and quantiles are all easy and robust — then average over directions. This is exactly aligned with how the energy score decomposes. For a uniformly random u on the sphere and any $x \in \mathbb{R}^d$,

$$\mathbb{E}_u |\langle u, x \rangle| = c_d \|x\|, \quad c_d = \frac{\Gamma(d/2)}{\sqrt{\pi} \Gamma((d+1)/2)},$$

so $\|x\| = c_d^{-1} \mathbb{E}_u |\langle u, x \rangle|$. Substituting into the energy score $\text{ES}(P, y) = \mathbb{E}\|X - y\| - \frac{1}{2}\mathbb{E}\|X - X'\|$ gives the projection identity

$$\boxed{\text{ES}(P, y) = c_d^{-1} \mathbb{E}_u [\text{CRPS}(P_u, \langle u, y \rangle)]} \quad (\text{PROJ})$$

where P_u is the law of the projected sample $\langle u, X \rangle$. The multivariate energy score *is* the average over random directions of the one-dimensional CRPS — and the energy score is strictly proper for the full distribution (Gneiting & Raftery 2007). The sliced quantity is therefore a proper score that needs only 1-D evaluations. We verify (PROJ) numerically: in [energy_score_via_projection](#) the sliced estimate matches the exact multivariate energy score within a few percent at a few thousand directions, and equals the CRPS exactly in 1-D ([test_nearest_the_pin.py](#)).

The projection-scored nearest-the-pin. Replace $q_i(z)$ in (NTP) by a projection-based skill score: for each direction u , the 1-D CRPS (or 1-D density) of participant i 's projected cloud at $\langle u, z \rangle$; average over u to get a per-participant score; split the pot in proportion to it. The pool keeps its self-funding and truthfulness properties (the energy score is proper, so truthful reporting is still optimal) while becoming computable and stable in high dimensions. This is, in spirit, the projection version at [monteprediction](#): score the eleven-dimensional cloud through its one-dimensional shadows.

Link to the random-projections literature. Slicing a high-dimensional problem into random 1-D projections is a recurring, theoretically-backed device:

- **Johnson–Lindenstrauss (1984):** random projections approximately preserve pairwise Euclidean distances — which is precisely the quantity the energy score / energy distance is built from, so a modest number of directions preserves the score.
- Sliced Wasserstein distances (Rabin et al. 2011; Bonneel et al. 2015): average 1-D optimal-transport costs over random projections, a now-standard, cheap surrogate for the multivariate Wasserstein distance — the optimal-transport cousin of (PROJ).
- Sliced score matching (Song et al. 2019): estimate high-dimensional score functions through random projections, for the same computational reasons.
- Energy distance (Székely & Rizzo 2013) is itself an integral of squared characteristic-function differences and, via the identity above, of absolute projected differences — the projection representation is intrinsic, not a heuristic.

The takeaway: the “projection version” is not an approximation bolted onto the mechanism — it is the *native* high-dimensional form of a density-pot-split parimutuel whose proper score (the energy score) is, by construction, an average over random projections.

5. High-dimensional joint scoring and the Schur pseudo-likelihood

Step back from the pool to the statistical problem underneath: **how do you score a joint distributional forecast in $\mathbb{R}^d$ when d is large relative to the data you can condition on?** This is

exactly the regime ($p > n$) in which the naïve held-out Gaussian likelihood — the density-based score — becomes unreliable, because the estimated covariance is rank-deficient and its inverse is nonsense.

The portfolio/spatial-statistics literature answers this from the density side. Cotton’s *Two Sides of Schur Damping* (2025) and the underlying Schur complementary allocation (Cotton 2024, arXiv:2411.05807) observe that a Gaussian density factorises through a Vecchia / conditional pseudo-likelihood $\prod_k \mathcal{N}(y_k; b_k^\top y_c, S_k)$ whose conditional covariances S_k are *Schur complements*, and that the reliable score in the undersampled regime is a damped version of this factorisation — the Schur pseudo-likelihood — with a closed-form James–Stein reliability damping γ^* . In other words, when the raw joint density is untrustworthy, score it through a structured, positive-definiteness-preserving factorisation rather than the full inverse covariance. (This is the basis of the *precise* library’s covariance assessors; see the [schur](#) project.)

The nearest-the-pin parimutuel needs precisely such a score: it must turn each participant’s joint forecast into a number at z . There are then two routes, and they are the two sides of the same coin:

route	how the joint forecast is scored	regime it suits
density	structured / damped joint density — Schur pseudo-likelihood, Vecchia factorisation	a parametric or covariance-shaped forecast; $p > n$ handled by damping
projection	average 1-D CRPS over random directions — the sliced energy score (PROJ)	a free-form sample cloud; high d handled by slicing

This is the paper’s organising claim: **the density route (Schur/Vecchia) and the projection route (energy/sliced) are alternative ways to make a joint forecast scoreable in high dimensions, and the nearest-the-pin parimutuel can be run with either.** A speculative but appealing synthesis — flagged as conjecture — is a *Schur-damped* projection score, in which the directions u are not isotropic but shaped by a damped estimate of the forecast covariance (project more often along the well-estimated directions), interpolating between the two columns with a single reliability dial γ exactly as in the Schur work. We leave its analysis open.

6. Why this matters: microprediction

The nearest-the-pin parimutuel is one of the mechanisms in the spirit of the microprediction vision (Cotton 2022): a web-scale network of autonomous forecasters continuously submitting *distributional* predictions and being paid by self-funding, truth-eliciting pools. The book surveys a range of such reward mechanisms; the nearest-the-pin pool is one concrete instance. Two further pieces close the loop:

- **Calibration via Z-streams.** The crowd’s aggregate density Q induces, for a scalar quantity, z -scores $\Phi^{-1}(F(x))$; if the market is calibrated these are standard normal over time, and any departure (fat tails, skew, autocorrelation) is an exploitable, self-correcting anomaly. The pool’s payouts push the aggregate back toward calibration.

- **Aggregation.** The crowd density Q is itself a forecast — a wealth-weighted pool of the participants’ densities, i.e. a (log-)opinion pool whose weights are endogenously set by past accuracy.

Together these make the nearest-the-pin parimutuel the connective tissue between the scoring-rule, parimutuel, and aggregation families mapped in the [relationship map](#).

7. Open questions

1. **Finite- b and finite- n truthfulness.** §3’s truthfulness argument is first-order in the wealth fraction b and assumes a small participant. What is the exact equilibrium for finite b and finitely many strategic participants? (The discrete DPM is known to admit non-truthful equilibria; cf. the projection game.)
2. **Choice of score.** Density pot-split (KDE) vs. projection (sliced energy) are both proper but reward different aspects of a forecast. Which yields better calibration / faster wealth concentration on skilled forecasters?
3. **Schur-damped projections (§5 conjecture).** Does anisotropic, covariance-shaped slicing with a reliability dial γ dominate isotropic slicing in the $p > n$ regime, and does it inherit the closed-form γ^* ?
4. **Variance of the sliced estimator.** How many directions are needed for the sliced score to rank participants correctly, as a function of d and the sample-cloud size? (A Johnson–Lindenstrauss-style bound.)

References

- Breiman, L. (1961). “Optimal Gambling Systems for Favorable Games.” *4th Berkeley Symposium*.
- Bonneel, N., Rabin, J., Peyré, G. & Pfister, H. (2015). “Sliced and Radon Wasserstein Barycenters of Measures.” *J. Math. Imaging Vis.* 51(1).
- Cotton, P. (2022). *Microprediction: Building an Open AI Network*. MIT Press.
- Cotton, P. (2024). “Schur Complementary Allocation.” arXiv:2411.05807.
- Cotton, P. (2025). “Two Sides of Schur Damping: High-Dimensional Pseudo-Likelihoods and Portfolio Allocation.” precise.microprediction.org.
- Gneiting, T. & Raftery, A. E. (2007). “Strictly Proper Scoring Rules, Prediction, and Estimation.” *JASA* 102(477).
- Hanson, R. (2007). “Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation.” *J. Prediction Markets* 1(1).
- Johnson, W. B. & Lindenstrauss, J. (1984). “Extensions of Lipschitz mappings into a Hilbert space.” *Contemp. Math.* 26.
- Kelly, J. L. (1956). “A New Interpretation of Information Rate.” *Bell System Technical Journal* 35(4).
- Pennock, D. (2004). “A Dynamic Pari-Mutuel Market for Hedging, Wagering, and Information Aggregation.” *ACM EC’04*.
- Rabin, J., Peyré, G., Delon, J. & Bernot, M. (2011). “Wasserstein Barycenter and its Application to Texture Mixing.” *SSVM*.
- Song, Y., Garg, S., Shi, J. & Ermon, S. (2019). “Sliced Score Matching.” *UAI*.

- Székely, G. & Rizzo, M. (2013). "Energy Statistics: A Class of Statistics Based on Distances." *J. Stat. Plan. Inf.* 143(8).
- Vecchia, A. V. (1988). "Estimation and Model Identification for Continuous Spatial Processes." *JRSS-B* 50(2).

Implementation: [mechanisms/nearest_the_pin.py](#). *Tests (self-funding, truthfulness, projection identity):* [tests/test_nearest_the_pin.py](#).